

# Data Structure Lab5 : Circularly Linked List 2022-2023

## Topics

1. Implement Node Class
2. Implement CircularlyLinkedList Class
3. Implement Basic Methods of CircularlyLinkedList
  - isEmpty()
  - size()
  - first()
  - last()
  - addFirst()
  - addLast()
  - removeFirst()
  - rotate()

## Homework

1. Consider the implementation of CircularlyLinkedList.addFirst, in Code Fragment 3.16. The else body at lines 39 and 40 of that method relies on a locally declared variable, newest. Redesign that clause to avoid use of any local variable.

```
public void addFirst(T element) {  
    if (isEmpty()) {  
        head = new Node<>(element);  
        head.next = head; // Point to itself  
    } else {  
        Node<T> newNode = new Node<>(element, head.next);  
        head.next = newNode; // Link the new node to the first node  
    }  
}
```

## Data Structure Lab5 : Circularly Linked List 2022-2023

2. Give an implementation of the size( ) method for the CircularlyLinkedList class, assuming that we did not maintain size as an instance variable.

```
public int size() {  
    if (isEmpty()) return 0;  
    int count = 1;  
    Node<T> current = head.next; // Start from the first node  
    while (current != head) {  
        count++;  
        current = current.next;  
    }  
    return count;  
}
```

3. Implement the equals( ) method for the CircularlyLinkedList class, assuming that two lists are equal if they have the same sequence of elements, with corresponding elements currently at the front of the list.

```
@Override  
public boolean equals(Object obj) {  
    if (!(obj instanceof CircularlyLinkedList)) {  
        return false;  
    }  
    CircularlyLinkedList<?> other = (CircularlyLinkedList<?>) obj;  
    Node<?> current1 = this.head;  
    Node<?> current2 = other.head;  
  
    if (current1 == null && current2 == null) return true; // Both empty  
    if (current1 == null || current2 == null) return false; // One is empty
```

## Data Structure Lab5 : Circularly Linked List 2022-2023

```
do {
    if (!current1.data.equals(current2.data)) {
        return false;
    }
    current1 = current1.next;
    current2 = current2.next;
} while (current1 != head && current2 != other.head);
return current1 == head && current2 == other.head; // Both
reached start
}
```

4. Suppose you are given two circularly linked lists, L and M. Describe an algorithm for telling if L and M store the same sequence of elements (but perhaps with different starting points).

-If the two lists are empty, they are equal.

- Set pointers currentL and currentM to the head of lists L and M, respectively.

- For each node in L:

- Compare the current nodes of L and M.
- If they are equal, move currentM to the next node.
- Continue this until you have compared all nodes in L.

5. Given a circularly linked list L containing an even number of nodes, describe how to split L into two circularly linked lists of half the size.

-Initialize two pointers, slow and fast, both starting at the head.

-Move fast two steps and slow one step until fast reaches the head:

- When fast reaches the head, slow will be at the midpoint.
- Set head1 to head, and head2 to slow.next.

## Data Structure Lab5 : Circularly Linked List 2022-2023

-Break the circular links:

- Set slow.next to head2.
  - Find the tail of the first half and set its next pointer to head1.
- Now you have two circularly linked lists.

6. Implement the clone( ) method for the CircularlyLinkedList class.

```
public CircularlyLinkedList<T> clone() {  
    CircularlyLinkedList<T> newList = new CircularlyLinkedList<>();  
    if (isEmpty()) return newList;  
  
    Node<T> current = head;  
    do {  
        newList.addLast(current.data); list  
        current = current.next;  
    } while (current != head);  
    return newList;  
}
```